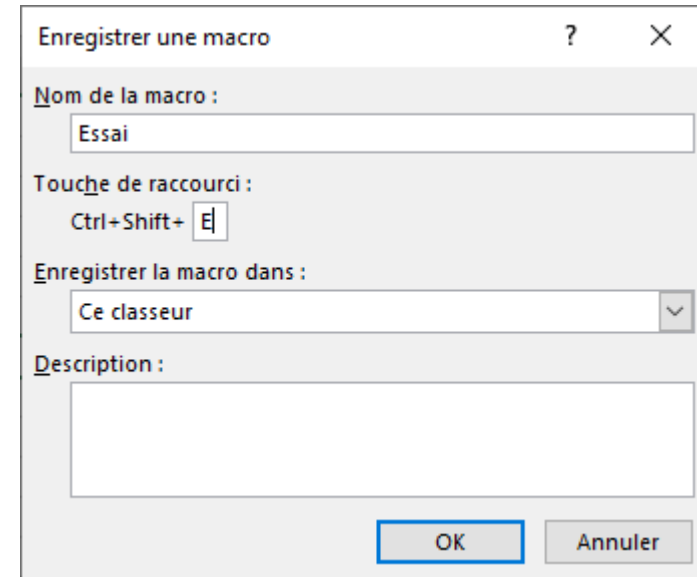


■ Macro et VBA

- Les macros
 - ✓ Définition
 - Suite d'instructions écrites l'une après l'autre
 - Permet d'automatiser une série de tâches à effectuer
 - C'est aussi un programme écrit en VBA (Visual Basic for Applications)
 - ✓ Principe de fonctionnement
 - On démarre l'enregistrement d'une macro
 - On effectue certaines tâches sur Excel
 - On arrête la macro
 - Ensuite on exécute autant de fois que l'on le désire la macro enregistrée pour répéter les tâches effectuées

■ Macro et VBA

- Les macros
 - ✓ Enregistrement d'une macro
 - Cliquer sur menu **Affichage** puis **Macros/Enregistrer une macro**
 - Donner un nom à la macro
 - Si possible une touche de raccourci
 - Cliquer sur **Ok** pour démarrer l'enregistrement de la macro
 - Les tâches de la macro
 - Cliquer sur une cellule et saisir l'information **Essai**
 - Cliquer sur une autre cellule et taper **"=Aujourd'hui()"**

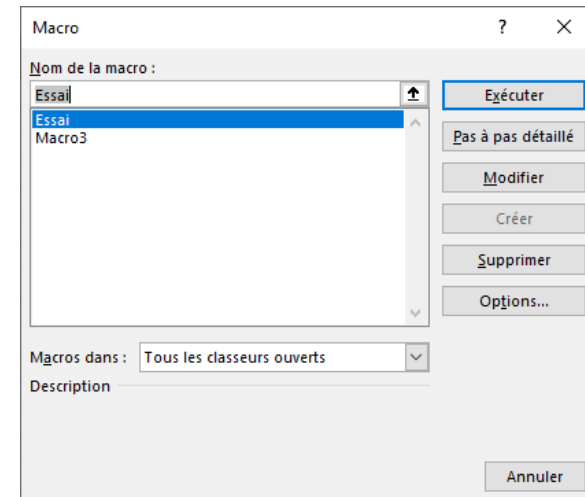


Excel avancé

■ Macro et VBA

- Les macros
 - ✓ Arrêter l'enregistrement d'une macro
 - Cliquer sur menu **Affichage** puis **Macros/Arrêter l'enregistrement**
 - ✓ Exécution d'une macro
 - Effacer les informations saisies lors de l'enregistrement de la macro
 - Cliquer sur **Affichage** puis **Macros/Afficher les macros**
 - Choisir la macro à exécuter
 - Puis cliquer sur **Exécuter**

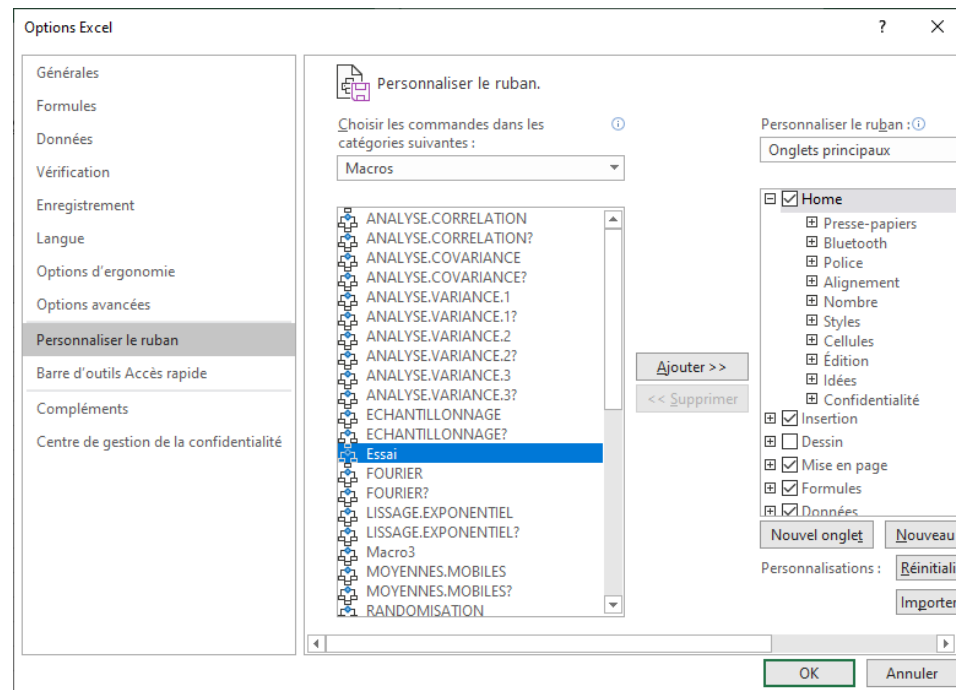
Les informations saisies sont réaffichées



Excel avancé

■ Macro et VBA

- Les macros
 - ✓ Lancer une macro à partir du ruban
 - Cliquer droit sur le ruban et choisir **Personnaliser le ruban**
 - Dans la liste déroulante choisir **Macros**
 - Choisir la macro puis cliquer sur **Ajouter**
 - Créer un **Nouveau groupe** si nécessaire



■ Macro et VBA

- Exercices
 - ✓ Créer une macro MACROMOIS qui écrit les noms de mois Janvier, Février et Mars en colonne à partir de la cellule active. Utiliser l'enregistreur de macro avec référence relative aux cellules.

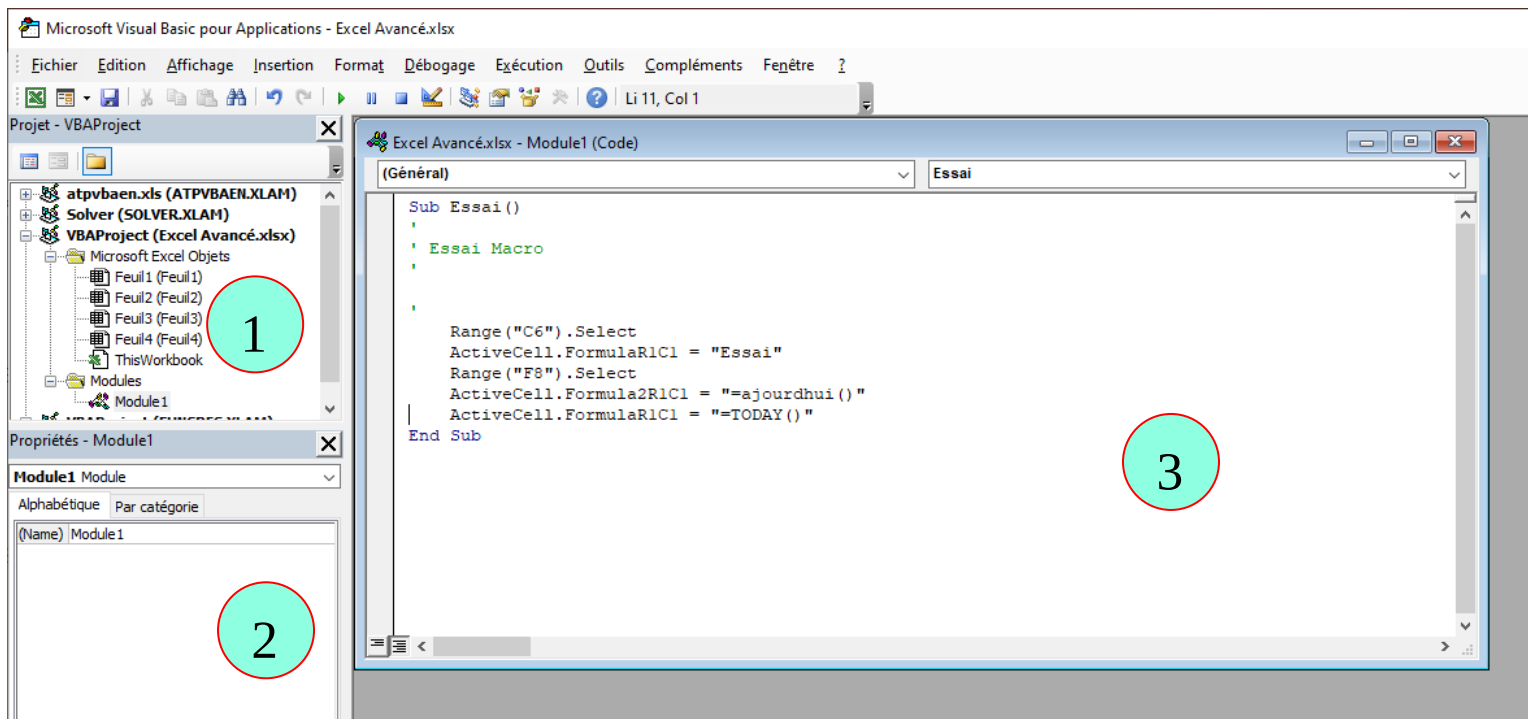
Excel avancé

■ Macro et VBA

- VBA

- ✓ Le code s'écrit dans un éditeur (VBE – Visual Basic Editor)

- Faire **ALT + F11** pour ouvrir l'éditeur



■ Macro et VBA

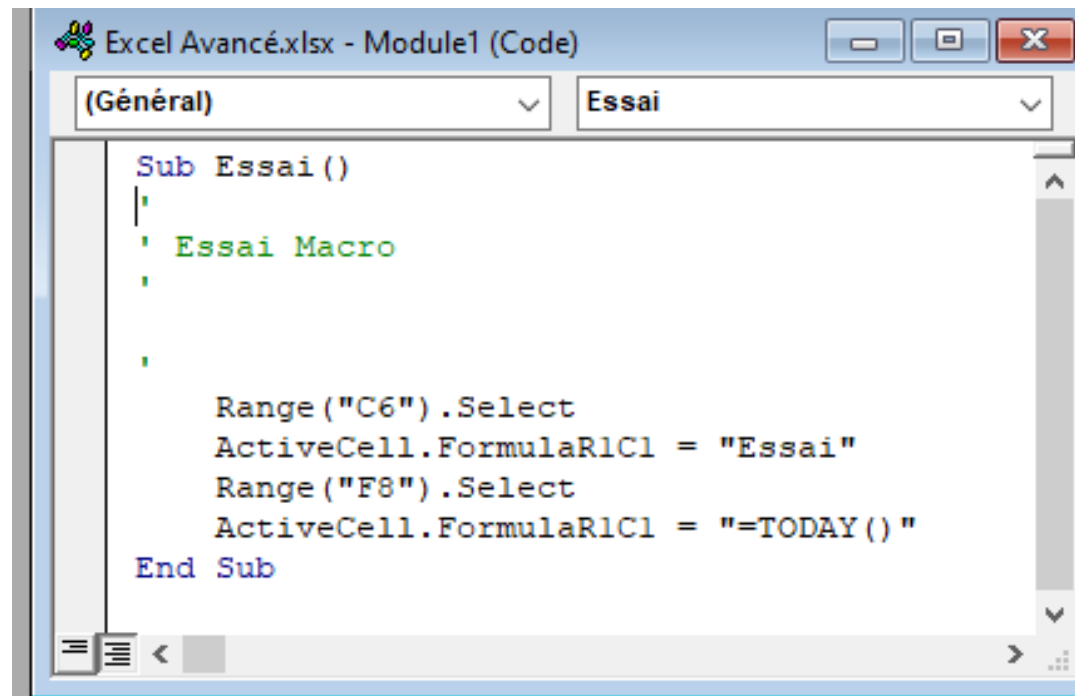
- VBA
 - ✓ Les différentes fenêtres du VBE
 - (1) **Explorateur de projet** : affiche une arborescence de chaque projet. Un projet correspond à l'ensemble du classeur (**Microsoft Excel Objects**), des feuilles de code (**Modules**) et éventuellement des boîtes de dialogue personnalisées (feuille ou **Userform**)
 - (2) **Propriétés** : permet de modifier les propriétés de chaque objet du projet. N.B. : VBA est un langage orienté objet. Les objets disposent donc de propriétés et méthodes
 - (3) **Code** : un projet peut contenir plusieurs feuilles de code qui s'affichent dans la zone de travail. En exemple, on peut voir le code de la macro Essai enregistrée précédemment dans un module.
 - On peut afficher d'autres fenêtres telle que la fenêtre d'exécution

■ Macro et VBA

- VBA

- ✓ Analyse du code de la macro Essai

- Les actions d'une macro se définissent dans une procédure (Sub). Donc Macro = Procédure
- Les actions se définissent avec les objets de Excel : Range – ActiveCell – etc.
- Range("C6").Select → sélection la cellule C6
- ActiveCell.FormulaR1C1 → affecte une valeur à la cellule active



```
Excel Avancé.xlsx - Module1 (Code)
(Général) Essai
Sub Essai()
' Essai Macro
'
Range("C6").Select
ActiveCell.FormulaR1C1 = "Essai"
Range("F8").Select
ActiveCell.FormulaR1C1 = "=TODAY()"
End Sub
```


■ Macro et VBA

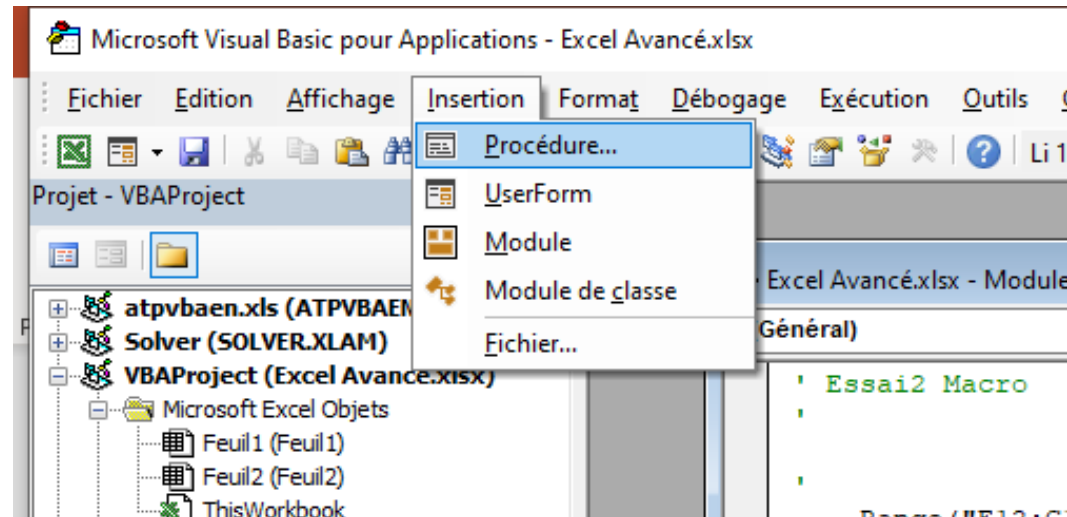
- VBA
 - ✓ Les objets dans Excel
 - **Application** : c'est Excel lui-même
 - **Workbooks** : les classeurs
 - **Sheets** : les feuilles
 - **Worksheets** : les feuilles de calcul
 - **Charts** : les feuilles graphiques
 - **Range** : une plage de cellule
 - **Cells** : les cellules
 - Par exemple, pour travailler sur la 1^{ère} cellule de la plage A10:B20 de la 1^{ère} feuille du classeur Test.xlsm, on écrira ceci :
`Workbooks("Test.xlsm").Worksheets(1).Range("A10:B20").Cells(1,1)`

Excel avancé

■ Macro et VBA

- VBA
 - ✓ La composition d'un programme
 - Procédure
 - UserForm
 - Module
 - Module de classe

Les procédures et fonctions se définissent dans des modules



■ Macro et VBA

- VBA : les éléments du langage
 - ✓ Opérateurs arithmétiques
 - `+`, `-`, `*` et `/` : opérateurs usuels sur les entiers et réels
 - `\` : division entière, `^` : à la puissance, `mod` : reste de la division
 - `&` : opérateur de concaténation pour les chaînes de caractères
 - ✓ Opérateurs de comparaison
 - `<`, `<=`, `>`, `>=`, `=` et `<>` : opérateurs usuels de comparaison
 - ✓ Opérateurs logiques
 - `And`, `Or`, `Eqv`, `Xor` et `Not`
 - ✓ N.B. : des priorités sont prédéfinies, pour forcer la priorité il faut utiliser des parenthèses

■ Macro et VBA

- VBA : les éléments du langage VBA
 - ✓ Les variables et constantes
 - Les types des variables
 - **Byte** (0 à 255), **integer** et **long** : pour les entiers
 - **Single** et **double** : pour les réels
 - **boolean** : **True** ou **False** (booléen)
 - **Date** : type date
 - **String** : chaînes de caractères (0 à 2 millions)
 - **Object** : n'importe quel objet
 - **Variant** : variable déclarée lors de son affectation, et peut passer d'un type à un autre

■ Macro et VBA

- VBA : les éléments du langage VBA
 - ✓ Les variables et constantes
 - Déclaration de variables et constantes
 - Syntaxe : `Dim|Private|Public|Static nom_variable As Type_variable`
 - **Dim** : dans une procédure ou Module portée locale
 - **Private** : dans un module portée locale
 - **Public** : dans un module, visible dans tout le classeur
 - **Static** : uniquement dans une procédure, garde sa valeur d'un appel à un autre
 - Constante : `Const Nom_constante As Type_constante = valeur`
 - **=** : instruction d'affectation de valeur

■ Macro et VBA

- VBA : les éléments du langage VBA
 - ✓ Les variables et constantes
 - Déclaration de variables objets
 - Syntaxe : `Dim nom_variable As Type_objet`
 - Exemples : `Dim UnClasseur As Workbook`
`Dim UneFeuille As Worksheet`
`Dim UneCellule As Range`
 - Affectation d'objet (à l'aide du mot clé `Set`)
 - Syntaxe : `Set nom_variable = nom_objet`
 - Exemples : `Set UnClasseur = ActiveWorkbook`
`Set UneFeuille = ActiveSheet`
`Set UneCellule = ActiveCell`

■ Macro et VBA

- VBA : les éléments du langage VBA
 - ✓ Les tableaux
 - Déclaration de tableaux
 - Syntaxe : `Dim nom_tableau(t1, t2, ...) As Type`
 - Exemples : `Dim iTab(10) As Integer`
`Dim dTable(4,5) As Double`
`Dim sTab (2,3,8) As String`
 - La numérotation commence par 0, à moins de le spécifier par :
`Dim Tab(1 To 5, 2 To 8) As Integer`
 - Déclaration dynamique de tableaux (mot clé `ReDim`)
 - Syntaxe : `Dim UnTableau As Type`
`ReDim UnTableau(taille)`

■ Macro et VBA

- VBA : les éléments du langage VBA
 - ✓ Interaction avec l'utilisateur
 - Affichage : `MsgBox` affiche boîte de dialogue
 - Syntaxe : `MsgBox message, [lesBoutons], [letitre]` ou
`nom_variable = MsgBox(message, [lesBoutons], [letitre])`
`lesBoutons` peuvent contenir icône et boutons à afficher
 - Constantes à additionner pour afficher icônes et boutons :
icônes : `vbCritical` – `vbExclamation` – `vbInformation` – `vbQuestion`
boutons : `vbOkCancel` – `vbYesNo` – `vbRetryCancel` –
`vbYesNoCancel` – `vbAbortRetryIgnore` – `vbDefaultButton2` –
`vbDefaultButton3`
 - Exples : `MsgBox "il est " & Format(Now, "hh:mm:ss"), , "Heure"`

■ Macro et VBA

- VBA : les éléments du langage VBA
 - ✓ Interaction avec l'utilisateur
 - Affichage : `MsgBox` affiche boîte de dialogue
 - `MsgBox` utilisé comme fonction renvoie un résultat de type entier correspondant au bouton sur lequel on clique :
 - `vbOk` : Ok – `vbYes` : Oui – `vbNo` : Non – `vbCancel` : Annuler – `vbAbort` : Abandonner – `vbRetry` : Recommencer – `vbIgnore` : Ignorer
 - Exples : `Reponse = MsgBox("Voulez-vous poursuivre ?", vbQuestion + vbYesNo + vbDefaultButton2, "Une petite question")`
`if Reponse = vbYes then`
...
...

■ Macro et VBA

- VBA : les éléments du langage VBA
 - ✓ Interaction avec l'utilisateur
 - Saisie : `InputBox` permet de saisir une valeur et retourne cette valeur après avoir cliquer sur bouton `Ok` ou chaine vide pour `Annuler`
 - Syntaxe : `nom_var = InputBox(message, [letitre], [valeur défaut])`
 - Expl : `Resultat = InputBox("Entrez le montant HT", "Prix TTC", 1)`
 - ✓ Les structures de contrôles
 - La séquence : `With ... End With`
 - Syntaxe : `With Nom_Obj`
 - `.nom_propriété = valeur`
 - `.nom_méthode(liste_param)`
 - `...`
 - `End With`

■ Macro et VBA

- VBA : les éléments du langage VBA

- ✓ Les structures de contrôles

- Les conditions : If ... Then ...

- Syntaxes : If condition Then

Instructions

...

[Else

Instructions

...]

End If

If cond Then Instruction

■ Macro et VBA

- VBA : les éléments du langage VBA

- ✓ Les structures de contrôles

- Les conditions : **Select ... Case ...**

- Syntaxe : **Select Case Expression**

Case valeur₁|expr₁

Instructions

- valeur_i : peut être constante ou variable, ou une énumération

- Expr_i : est de la forme Is **op** val₁ ou val₁ **To** val₂

avec op : <, <=, >, >=

...
Case valeur_i|Expr_i

Instructions

[Case Else

Instructions]

End Select

■ Macro et VBA

- VBA : les éléments du langage VBA
 - ✓ Les structures de contrôles
 - Les boucles : `For ... Next`
 - Syntaxe : `For compteur = val1 To val2 [Step pas]`
Instructions
...
`Next [compteur]`
 - Les boucles : `For Each ... Next`
 - Syntaxe : `For Each variable_objet In Collection`
Instructions
...
`Next [variable_objet]`

■ Macro et VBA

- VBA : les éléments du langage VBA

- ✓ Les structures de contrôles

- Les boucles : Do ... Loop

- Syntaxes :

Do
Instructions
...
[If cond Then Exit Do]
Instructions
...
Loop While Condition

Do Until Condition
Instructions
...
[If cond Then Exit Do]
Instructions
...
Loop

Do While Condition
Instructions
...
[If cond Then Exit Do]
Instructions
...
Loop

Do
Instructions
...
[If cond Then Exit Do]
Instructions
...
Loop Until Condition

■ Macro et VBA

- VBA : les éléments du langage VBA
 - ✓ Les structures de contrôles
 - Les boucles : **Do ... Loop** ou **While ... Wend**
 - Syntaxes :

```
Do
  Instructions
  ...
  [If condition Then Exit Do]
  Instructions
  ...
Loop
```

```
While Condition
  Instructions
  ...
Wend
```

■ Macro et VBA

- VBA : les éléments du langage VBA
 - ✓ Les sous programmes
 - Les procédures : `Sub ... End Sub`
 - Syntaxes : `[portée] Sub Nom_procedure([var As type, ...])`
`Instructions`
`...`
`End Sub`
 - La portée : `Dim`, `Public` ou `Private`
 - L'appel d'une procédure s'effectue dans une autre procédure de la manière suivante : `call Nom_procedure([liste_param])`

■ Macro et VBA

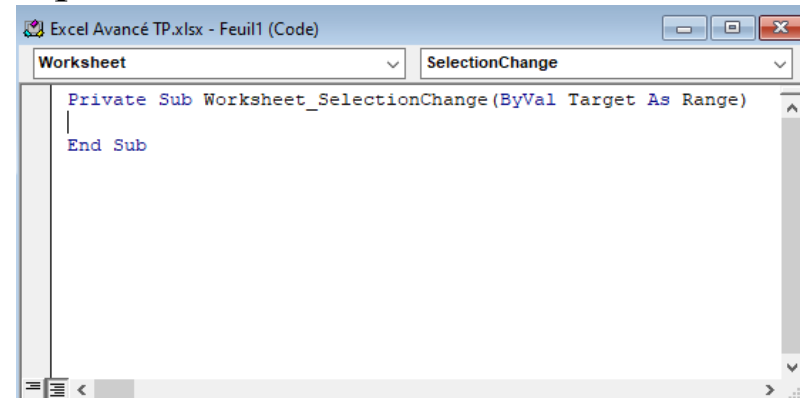
- VBA : les éléments du langage VBA
 - ✓ Les sous programmes
 - Les fonctions : `Function ... End Function`
 - Syntaxes : `[portée] Function Nom_fonction([liste_param]) As Type`
`Instructions`
`...`
`' avec instructions renvoyant le résultat`
`Nom_Fonction = lerésultat`
`End Function`
 - L'appel d'une fonction s'effectue soit directement dans le classeur à l'aide de l'insertion de fonction soit dans une autre procédure de la manière suivante : `variable = Nom_fonction([listes_param])`

Excel avancé

■ Macro et VBA

- VBA : les éléments du langage VBA
 - ✓ Les sous programmes
 - Les procédures événementielles : `Private Sub ... End Sub`
 - Elle est liée à un événement
 - L'ossature est donné par Excel
 - On écrit le code juste à l'intérieur de l'ossature
 - Elle s'exécute automatiquement quand l'événement survient

On l'ouvre en double-cliquant sur un objet Excel dans l'Explorateur de projets pour afficher sa feuille de code



■ Macro et VBA

- VBA : les objets d'Excel
 - ✓ Cellules et plages de cellules
 - Cellule objet Excel le plus manipulé : on le désigne par `Range` ou `Cells`
 - `Range` : représente une plage contenant une à plusieurs cellules
 - `Range("nom_plage")` ou `Range(référence_plage)`
 - `Cells(n°_ligne, n°_colonne)` désigne une cellule unique
 - Méthode `Evaluate("nom_à_convertir")` convertir un nom en un objet cellule ou valeur, on utilise souvent la forme `[nom_à_convertir]` par exemple : `[A1]` la cellule A1 ou `[cellule_nommée]`
 - `ActiveCell` : la cellule active

■ Macro et VBA

- VBA : les objets d'Excel
 - ✓ Cellules et plages de cellules
 - Se référer à une plage

code	description
Range("Data")	Plage nommée Data
Range(A1:C10)	Plage de A1 à C10
[A1:C10]	Plage de A1 à C10
Range(Cells(1,1), Cells(10,3))	Plage A1 à C10
Range(Range("A1"), Range("C10"))	Plage A1 à C10
Range("A1:B2;C10:D12")	Union des plages A1:B2 et C10:D12
Selection	Les cellules sélectionnées

■ Macro et VBA

- VBA : les objets d'Excel
 - ✓ Cellules et plages de cellules particulières
 - `Worksheets(1).Cells` l'ensemble des cellules de la 1^{ère} feuille
 - `Range("A1:C10").Cells` l'ensemble des cellules de la plage A1:C10
 - `Range("A1:C10").Columns` l'ensemble des colonnes de la plage
 - `Range("A1:C10").Rows` l'ensemble des lignes de la plage
 - `Worksheets(1).UsedRange` l'ensemble des cellules occupées
 - Méthode `SpecialCells` renvoie une partie des cellules de la plage indiquée elle prend en paramètre une valeur constante :
 - `xlCellTypeBlanks` pour les cellules vides
 - `xlCellTypeConstants` : cellules contenant des constantes
 - `xlCellTypeFormulas` : cellules contenant des formules

■ Macro et VBA

- VBA : les objets d'Excel
 - ✓ Cellules et plages de cellules particulières
 - Méthode `SpecialCells`
 - `xlCellTypeLastCell` : dernière cellule dans la plage utilisée
 - `xlCellTypeVisible` : toutes les cellules visibles
 - Exemple : sélection plage contenant les cellules vides de B2:D11
`Range("B2:D11").SpecialCells(xlCellTypeBlanks).Select`
 - Propriété `Offset(nbre_ligne, nbre_col)` : décale de `nbre_ligne` en dessous et `nbre_col` à droite, si valeur négative c'est dans l'autre sens
 - Exemple : sélection de la cellule située 1 ligne en-dessous et 2 colonnes à droite de la cellule A1
 - `Range("A1").Offset(1, 2).Select`

■ Macro et VBA

- VBA : les objets d'Excel
 - ✓ Modifier cellule ou plage
 - Modifier ou lire le contenu : propriété implicite **Value**, Exemples
 - `Range("B2").Value = "Assalé" – Range("C2") = "Louis" – Range("D2:D5") = 20`
 - Modifier ou lire la formule : 4 propriétés permettent de renvoyer la formule d'une cellule – différence au niveau de la langue et référence
Exemple : soit à faire la somme = SOMME(A2:C2) on peut écrire
 - `Range("D2").Formula = "=SUM(A2:C2)" –`
`Range("D2").FormulaLocal = "=SOMME(A2:C2)" –`
`Range("D2").FormulaR1C1 = "=SUM(RC[-3]:RC[-1])" –`
`Range("D2").FormulaR1C1Local = "=SOMME(LC(-3):LC(-1))"`
 - Il existe des méthodes pour le format de cellule : à rechercher

■ Macro et VBA

- VBA : les objets d'Excel
 - ✓ Manipuler une plage
 - Copier et coller : copie la cellule active et la colle une ligne en-dessous
 - `ActiveCell.Copy ActiveCell.Offset(1, 0)`
 - Copier : `ActiveCell.Copy`
 - Coller le format : `ActiveCell.Offset(2, 0).PasteSpecial xlPasteFormats`
 - Coller la valeur : `ActiveCell.Offset(3, 0).PasteSpecial xlPasteValues`
 - Coller formule : `ActiveCell.Offset(4, 0).PasteSpecial xlPasteFormulas`
 - Déplacer : `ActiveCell.Cut ActiveCell.Offset(1, 0)`
 - Effacer méthode : `Clear` tout – `ClearContents` le contenu – `ClearFormat` – `ClearComments` – `ClearHyperlinks` liens hypertexte
 - Supprimer méthode : `Delete Shift:xlUp` supprime et décale vers le haut

■ Macro et VBA

- VBA : Exercices

- La macro MACROMOIS doit avoir à peu près le code suivant :

```
Sub MacroMois()
```

```
    ActiveCell.FormulaR1C1 = "Janvier"
```

```
    ActiveCell.Offset(1, 0).Range("A1").Select
```

```
    ActiveCell.FormulaR1C1 = "Février"
```

```
    ActiveCell.Offset(1, 0).Range("A1").Select
```

```
    ActiveCell.FormulaR1C1 = "Mars«
```

```
End Sub
```

L'instruction `ActiveCell.Offset(1,0).Range("A1").Select` sélectionne (Select) une cellule (Range) qui est sur une ligne en dessous et sur la même colonne (`Offset(déplacement_ligne, déplacement_colonne)`) que la cellule active (`ActiveCell`).

■ Macro et VBA

- VBA : Exercices
 - L'instruction `ActiveCell.Offset(1,0).Range("A1").Select` peut être remplacée par l'instruction `ActiveCell.Offset(1,0).Select`.
Modifier la macro pour que le nom de mois **Avril** soit écrit, en caractères gras et italiques, dans la cellule qui se trouve dans la colonne de droite et sur la même ligne que la cellule où est écrit Janvier.

Janvier	<i>Avril</i>
Février	
Mars	

■ Macro et VBA

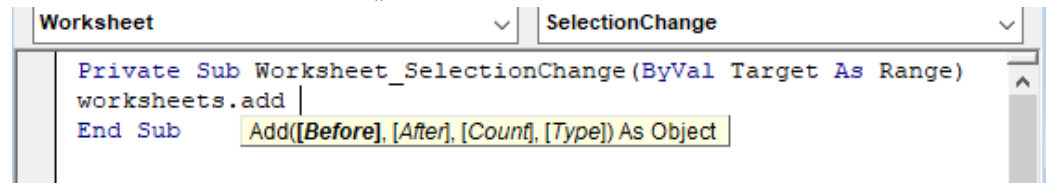
- VBA : les objets d'Excel
 - ✓ Les feuilles
 - 2 types de feuilles : feuille de calcul et feuille graphique
 - Collection `Sheets` contient l'ensemble des feuilles (tous les types)
 - Exemple : `Range(A1) = Sheets.Count` donne le nombre de feuille
 - Objet `Worksheet` de collections `Worksheets` utilisé pour feuille de calcul
 - Collection `Charts` représente l'ensemble des feuilles graphiques
 - Collection `ChartObjects` les graphiques situés sur une feuille de calcul
 - Se référer à une feuille
 - Syntaxe : `Worksheets(num_feuille) – Worksheets("Nom_feuille")`
 - Exemples : `Worksheets(Test).Activate – Ma_feuille.Activate`

■ Macro et VBA

- VBA : les objets d'Excel

- ✓ Ajouter une feuille : méthode **Add()**

- Les paramètres :



- **Before**, **After** : avant ou après une feuille

- **Count** : nombre de feuilles à ajouter

- Exemple : **Worksheets.Add Before:=Worksheets(4), Count:=2** ajoute 2 feuilles après la feuille 4

- ✓ Copier ou déplacer une feuille : méthodes **Copy()** ou **Move()**

- Exemples : **Worksheets("Data").Move Before:=Worksheets(1)**
Worksheets("Data").Copy After:=Worksheets("Data")

■ Macro et VBA

- VBA : les objets d'Excel
 - ✓ Supprimer une feuille : méthode `Delete()`
 - Exemple : `Worksheets("Data").Delete`
 - ✓ Renommer ou colorier un onglet
 - Renommer : propriété `Name`, Exemple : `ActiveSheet.Name = "Bilan"`
 - Colorier l'onglet d'une feuille, Exemple :
 - `ActiveSheet.Tab.ThemeColor = xlThemeColorAccent2`
`ActiveSheet.Tab.TintAndShade = 0.5`
 - ✓ Imprimer une feuille : méthode `PrintOut()`
 - Exemple : `Worksheets(Data).PageSetup.Orientation = xlLandscape`
`Worksheets(Data).PrintOut Copies:=2`

■ Macro et VBA

- VBA : les objets d'Excel
 - ✓ Les classeurs
 - Collection **Workbooks** : ensemble des classeurs ouverts dans Excel
 - Se référer à un classeur
 - **Workbooks(1) – Workbooks("nom_classeur")**
 - Objet **ActiveWorkbook** représente le classeur actif et **ThisWorkbook** le classeur contenant le code en cours d'exécution, Exemples :
 - **WorkBooks("Classeur.xlsm").Activate**
Range(A1) = WorkBooks(1).Name
Range(A2) = ActivateWorkbook.FullName
Range(A3) = ThisWorkbook.Path

■ Macro et VBA

- VBA : les objets d'Excel
 - ✓ Manipuler les classeurs
 - Méthode `Add()` : crée un nouveau classeur
 - Exemples : `Workbooks.Add` 'Nouveau classeur vierge'
`Workbooks.Add Template:="C:\sources\monModele.xlsx"`
nouveau classeur à partir du fichier monModele
 - Ouvrir et Fermer un classeur : méthodes `Open()` et `Close()`
 - `WorkBooks.Open "C:\Mesclasseurs\Classeur.xlsx"`
 - `Close` prend un paramètre : `True` ou `False` (enregistrer ou pas)
`WorkBooks.Close False`
 - Enregistrer et Enregistrer sous : méthodes `Save()` et `SaveAs()`
 - `Workbooks.SaveAs "C:\Mesclasseurs\Classeur.xlsx"`
 - `ActivateWorkbook.Saved = True` ' marque le fichier comme sauvé

■ Macro et VBA

- VBA : les objets d'Excel
 - ✓ Les graphiques
 - Sur une feuille de graphique, objet **Chart** de la collection **Charts**
 - Sur feuille de calcul, objet **ChartObject** de la collection **ChartObjects**
 - Objet **ChartObject** contient lui-même un Objet **Chart**
 - Propriétés et méthodes de **ChartObject** gèrent l'aspect et la taille du conteneur, celles de **Chart** gèrent le graphique lui-même
 - ✓ Créer graphique méthode **Add()** :
 - Propriétés : **ChartType** (type) –
HasTitle (affichage titre) –
ChartTitle.Characters.Text (intitulé) –
SetSourceData (source du graphe) –
Name (le nom de la feuille)

```
Dim leGraph As Chart
Dim rPlage As Range
Set rPlage = flVentes.Range(C3:E15)
Set leGraph = Chart.Add
With leGraph
    .ChartType = xlColumnClustered
    ...
End With
```


■ Macro et VBA

- VBA : les objets d'Excel
 - ✓ Graphique incorporé :

Propriétés : **Axes** (description des axes) – **Legend.Position** (emplacement de la légende) – **Parent.Name** (nom de l'objet graphique - ChartObject

```
Dim leGraph As Chart
Dim rPlage As Range
With flVentes
    Set rPlage = .Range("C3:E15").Offset(0, 3)
    Set leGraph = ChartObjects.Add(rPlage.Left, _
        rPlage.Top, rPlage.Width, rPlage.Height).Chart
    Set rPlage = . Range("C3:E15")
End With
With leGraph
    .ChartType = xlBarStacked
    .SetSopurceData Source:=rPlage, PlotBy:xlColumns
    .HasTitle = False
    ...
End With
```

■ Macro et VBA

- VBA : les objets d'Excel
 - ✓ Graphique les axes :

Collection **Axes** : ensemble des axes, **Axis** est 1 objet de **Axes**.

Pour l'utiliser on précise :
son type : **xlValue** (axe des valeurs),
xlCategory (axe des abscisses) ou
xlSeriesAxis (pour les graphiques 3D)
et son groupe : **xlPrimary** ou **xlSecondary**
Propriétés : **MaximumScale** (valeur maximale de l'échelle), **MinimumScale** (valeur minimale de l'échelle)

```
Dim leGraph As Chart, Axe As Axis
Set leGraph = Charts("Graph")
With leGraph
    Set Axe = .Axes(xlCategory, xlPrimary)
    With Axe
        .HasTitle = True
        .AxisTitle.Text = "Années"
        ...
    End With
    Set Axe = .Axes(xlValue, xlPrimary)
    With Axe
        .HasTitle = False
        .MaximumScale = Valeur_Max_Echele
        .MinumumScale = Valeur_Min_Echelle
        ...
    End With
End With
```

■ Macro et VBA

- VBA : les objets d'Excel
 - ✓ Graphique les séries :

SeriesCollection : l'ensemble des séries, objet **Series** est une série du graphe.

Propriétés : **Format.Fill.ForeColor.RGB** (couleur de fond), **Format.Line.Visible** (bordure visibilité), **HasDataLabels** (ajout d'étiquette),

DataLabels.Font.Color (couleur police d'étiquette), **DataLabels.Font.Bold** (mis en gras d'étiquette)

```
Dim leGraph As Chart, UneSerie As Series
Set leGraph = Charts("Graph")
Set UneSerie = leGraph.SeriesCollection("Hommes")
With UneSerie
    .Format.Fill.ForeColor.RGB = RGB(67, 102, 180)
    .Format.Line.Visible = False
    .HasDataLabels = True
    ...
End With
Set UneSerie = leGraph.SeriesCollection("Femmes")
With UneSerie
    .Format.Fill.ForeColor.RGB = RGB(67, 102, 180)
    .Format.Line.Visible = False
    .HasDataLabels = True
    .DataLabels.Font.Color = RGB(255, 175, 2)
    ...
End With
```

■ Macro et VBA

- VBA : les objets d'Excel
 - ✓ Excel lui-même est l'objet `Application`
 - ✓ Modifier les options d'Excel, propriétés :
 - `Calculation` : modifie le mode de calcul : `xlCalculationAutomatic`, `xlCalculationManual`, `xlCalculationSemiautomatic`
 - `DisplayAlerts` à `False` supprime les messages d'avertissements
 - `ScreenUpdating` à `False` désactive la mise à jour de l'écran
 - `EnableEvents` à `False` bloque les événements
 - ✓ Utiliser les fonctions de calcul d'Excel
 - Syntaxe : `variable = Application.WorkSheetFunction.nom_fonc(param)`
 - Expl : `Range("B2") = Application.WorkSheetFunction.SUM("B5:E13")`

■ Macro et VBA

- VBA : les objets d'Excel
 - ✓ Utiliser les boîtes de dialogues
 - Ouvrir ou enregistrer un fichier : méthodes **GetOpenFilename** (affiche la boîte de dialogue Ouvrir) et **GetSaveAsFilename** (boîte de dialogue Enregistrer Sous) – s'utilisent avec **Open** et **SaveAs** de l'objet classeur

```
Sub OuvreFichier
    Dim Fichier As Variant
    Fichier = Application.GetOpenFilename
    If Fichier <> False Then Workbooks.Open Fichier
End Sub
```

```
Dim Fichier As Variant
Fichier = Application.GetSaveAsFilename
If Fichier <> False Then
    ActiveWorkbook.SaveAs Fichier
End If
```

- Demander une valeur à l'utilisateur : **Application.InputBox**
 - 3 paramètres dont l'1 le Type peut prendre valeurs : **0** (formule) – **1** (nombre) – **2** (texte) – **4** (valeur logique) – **8** (référence de cellules)

Expl : `Range("A1") = Application.InputBox(Prompt:="Entrer un nombre", Title:="Saisie", Type:=1)` 45

■ Travaux Pratiques

- Exercices
 - ✓ Ecrire l'instruction qui permet de faire référence à la cellule B2 dans la deuxième feuille de calcul de nom FEUIL2 du classeur TEST-MACRO.XLS.
 - ✓ Ecrire l'instruction qui permet d'affecter la valeur de la cellule B2 de la deuxième feuille de calcul du classeur TEST-MACRO.XLS à la cellule active.
 - ✓ Ecrire une macro qui permet de calculer la surface d'un rectangle (hauteur*largeur) à partir des valeurs contenues dans les cellules A1 et B1 de la deuxième feuille de calcul. Le résultat sera écrit dans la cellule C1.

■ Travaux Pratiques

- Exercices des Fichiers suivants :
 - ✓ EXCEL.B : exo 1, 2 et 3
 - ✓ EXCEL.D : exo 1 et 2
 - ✓ EXCEL.F : exo 4, 5, 7 et 8
 - ✓ EXCEL.G : exo 1
 - ✓ EXCEL.H : exo 1 et 2

Excel avancé

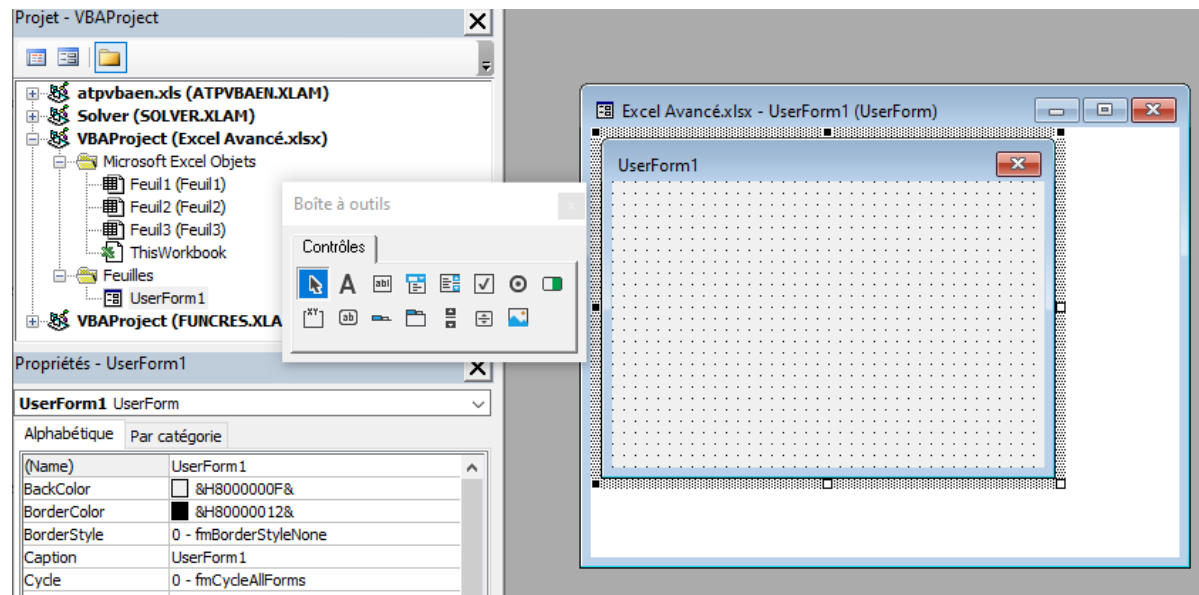
■ Interface graphique

- Les UserForm

- ✓ UserForm est l'interface graphique de VBA. C'est un formulaire sur lequel on peut déposer des contrôles

- Cliquer sur le menu Insertion puis UserForm

Un objet UserForm1 apparaît avec une boîte à outils



Excel avancé

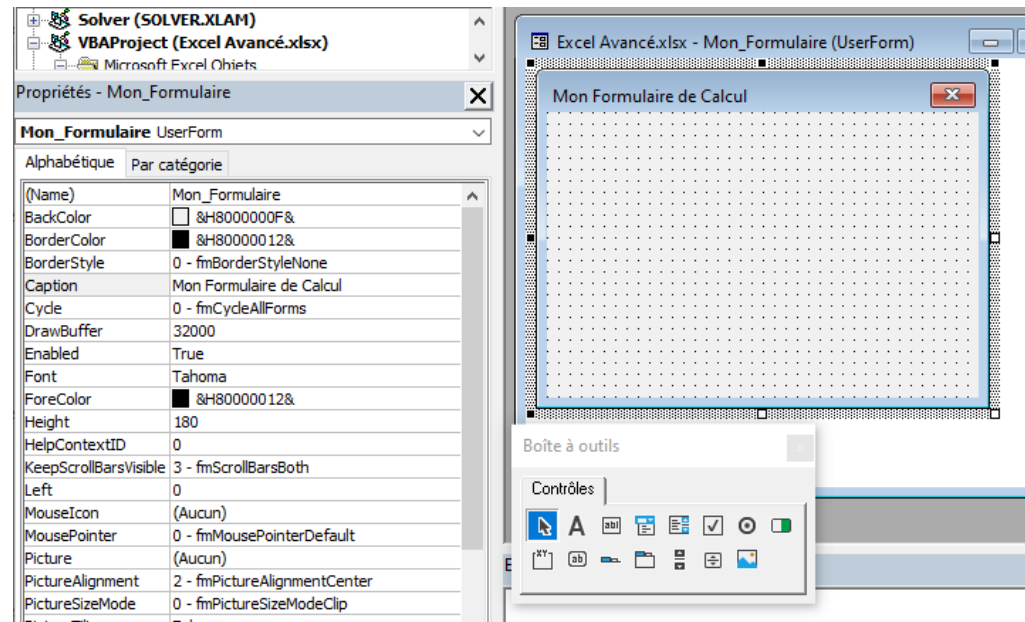
■ Interface graphique

- Les UserForm

- Les propriétés du UserForm apparaissent dans la fenêtre de propriétés

On peut changer le nom par défaut UserForm1 via la propriété (Name) et le titre via la propriété Caption

N.B. : la propriété Name donne le nom de l'objet et c'est à partir du nom qu'on peut accéder aux propriétés et méthodes de l'objet. La syntaxe est : `nom_objet.nom_prop = valeur`



Exemple : `Mon_Formulaire.Caption = "Mon formulaire de Calcul"`

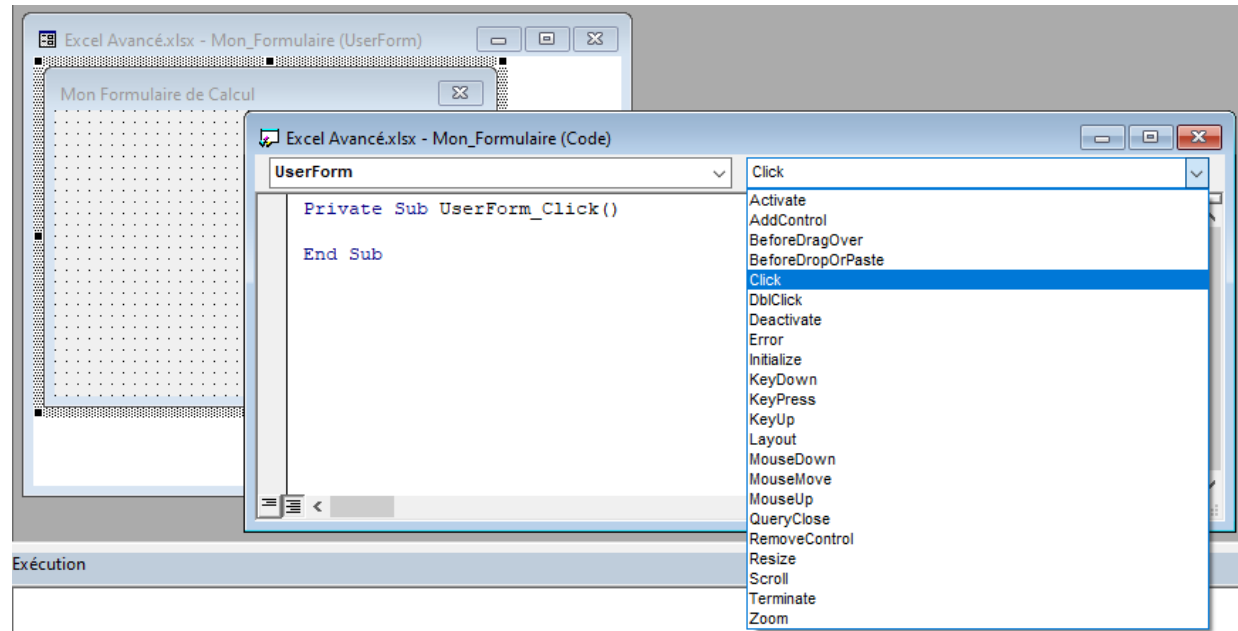
Excel avancé

■ Interface graphique

- Les UserForm
 - Les événements d'un UserForm en double-cliquant sur lui

La fenêtre du code des événements comprend 2 listes : liste de gauche → pour les noms des objets et la liste de droite → pour les événements liés à l'objet choisit dans la liste 1

N.B. : pour afficher les événements liés à un objet on double-clique sur l'objet



Excel avancé

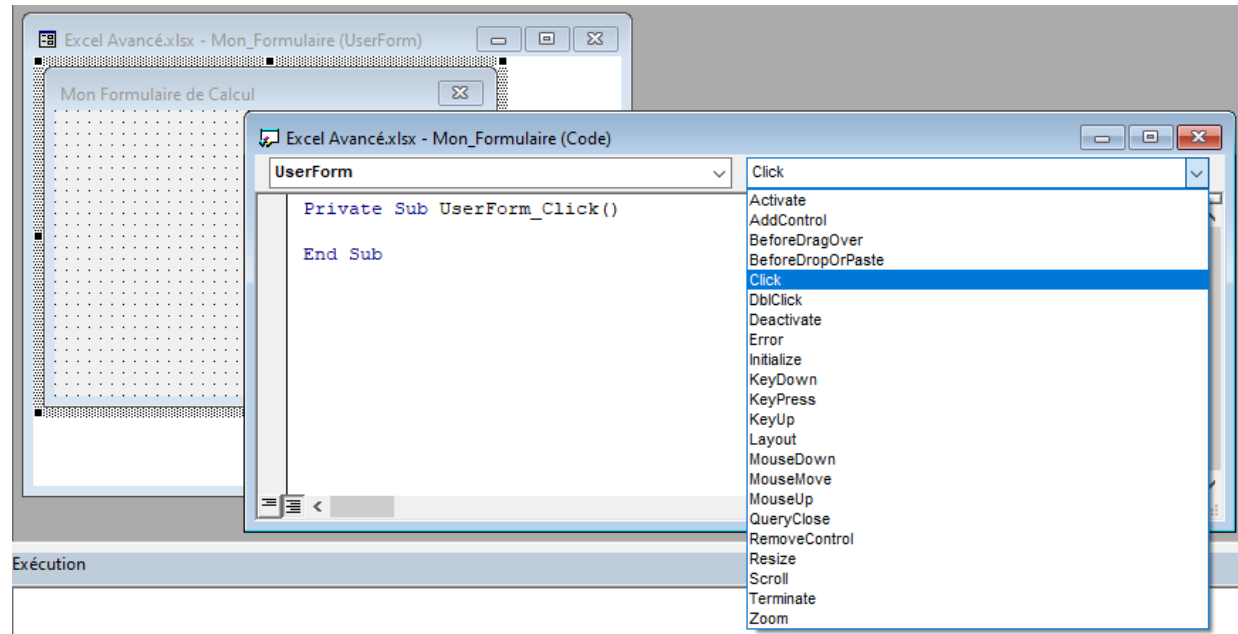
■ Interface graphique

- Les UserForm

- Afficher les événements d'un UserForm en double-cliquant sur lui

La fenêtre du code des événements comprend 2 listes : liste de gauche → pour les noms des objets et la liste de droite → pour les événements liés à l'objet choisit dans la liste 1

N.B. : pour afficher les événements liés à un objet on double-clique sur l'objet



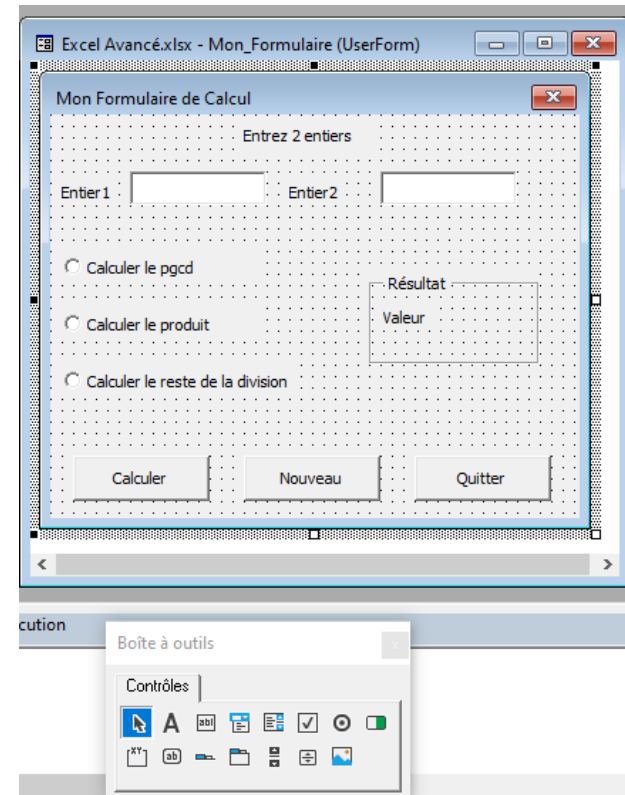
■ Interface graphique

- Les UserForm
 - Ajout de contrôles à un Userform

On clique sur le contrôle dans la boîte à outils et par glisser-déposer on le place sur le UserForm à notre convenance

Les contrôles ont des propriétés communes :

Name (nom interne) – **Caption** (libellé) – **TabIndex** (rang d'accessibilité sur le Userform) – **Enabled** (actif ou pas) – **Visible** (visible ou pas)

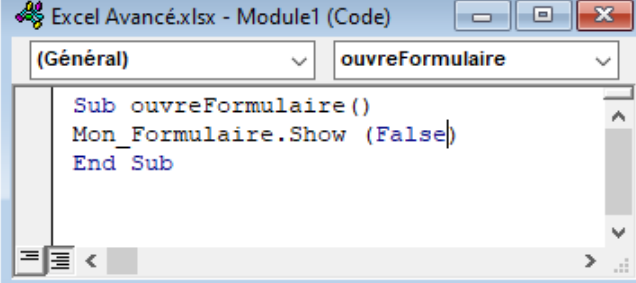


■ Interface graphique

- Les UserForm

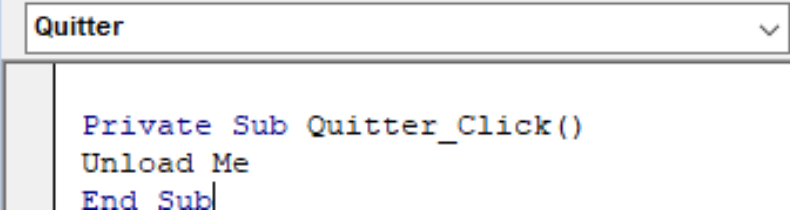
- Lancer/Fermer un Userform

- Pour lancer, on crée un module dans lequel on crée une procédure
 - Dans la procédure on utilise la méthode **Show** avec son paramètre à False (feuille est non modale)
 - Pour Fermer, dans événement Click d'un Bouton de commande on utilise l'instruction **Unload**
 - L'instruction **Load** charge une feuille en m'moire
 - La méthode **Hide** cache une feuille qui est affichée



The screenshot shows the VBA Code window for 'Excel Avancé.xlsx - Module1 (Code)'. The 'Général' tab is selected, and the procedure 'ouvreFormulaire' is chosen from the dropdown. The code is as follows:

```
Sub ouvreFormulaire()  
    Mon_Formulaire.Show (False)  
End Sub
```

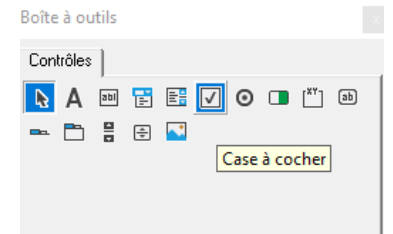
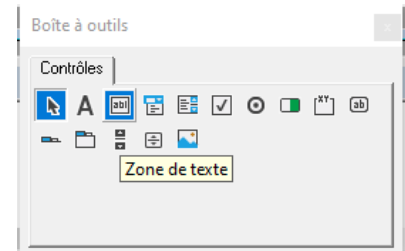
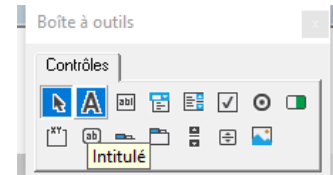


The screenshot shows the VBA Code window for a form named 'Quitter'. The 'Quitter' dropdown is selected, and the procedure 'Quitter_Click' is chosen. The code is as follows:

```
Private Sub Quitter_Click()  
    Unload Me  
End Sub
```

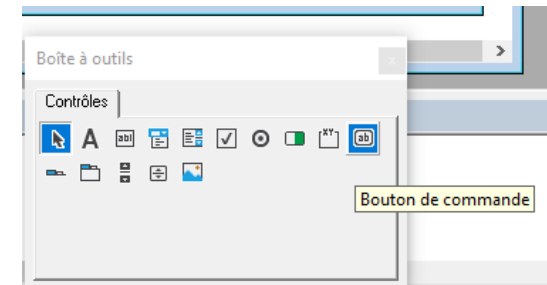
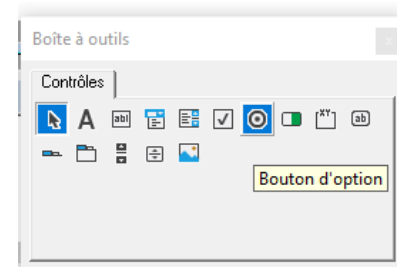
■ Interface graphique

- Les contrôles usuels
 - ✓ L'intitulé (**Label**)
 - Il sert à afficher des informations sur le UserForm
 - ✓ La zone de Texte (**TextBox**)
 - Il sert à saisir du texte
 - Il dispose de 2 événements importants :
 - **Change** qui s'exécute qd le contenu change
 - **KeyPress** qui s'exécute quand on appuie sur une touche
 - ✓ Case à cocher (**CheckBox**)
 - La propriété **Value** à **True** la case est cochée
 - Événements importants : **Click** et **Change**



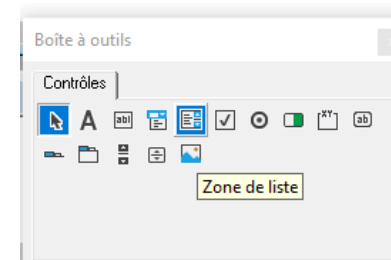
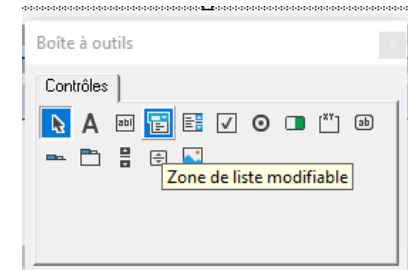
■ Interface graphique

- Les contrôles usuels
 - ✓ Les boutons d'option (**OptionButton**)
 - Il y en a au moins 2 mais 1 seul est sélectionné
 - Propriété **Value** à **True** sélectionne le contrôle
 - Événements importants : **Click** et **Change**
 - ✓ Le bouton de commande (**CommandButton**)
 - Il sert à exécuter des commandes
 - Événements importants : **Click** et **DblClick**
 - ✓ Image (**Image**)
 - Il sert à insérer une image
 - Propriété **Picture** permet de recevoir l'image



■ Interface graphique

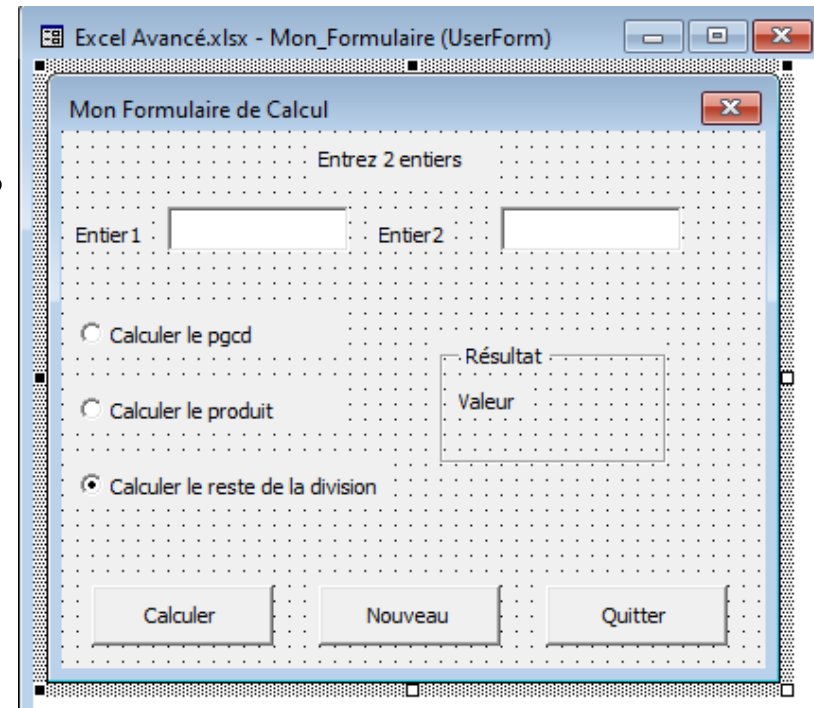
- Les contrôles usuels
 - ✓ La zone de liste modifiable (**ComboBox**)
 - C'est une liste déroulante (1 seul élément visible)
 - La méthode **AddItem** a 2 paramètres, ajoute un élément à une position spécifiée
`ComboBox1.AddItem("toto", 4)`
 - La méthode **RemoveItem** a 1 paramètre, supprime l'élément dont le rang est passé en paramètre. La méthode **Clear** efface tous les éléments
 - Événement important : **Click** et **Change**
 - ✓ La zone de liste (**ListBox**)
 - Plusieurs éléments visibles
 - Les mêmes méthodes et événements que le **ComboBox**



■ Travaux pratiques

- Exercice 1

1. Dessiner l'interface; changer la propriétés Name des objets suivants
textBox1 en Entier1 – textBox2 en Entier2 – OptionButton1 en pgcd – OptionButton2 en produit – OptionButton3 en reste – valeur en Valeur –
CommandButton1 en Calculer –
CommandButton2 en Nouveau et
CommandButton3 en Quitter
2. Écrire le code de sorte que si des valeurs sont saisies et une option choisie, à l'appui sur le bouton Calculer la valeur s'affiche dans le label Valeur



■ Travaux pratiques

- Exercice 1 : suite
 - 3. Écrire du code de sorte que quand l'on clique sur nouveau les valeurs s'effacent et le curseur est positionné dans Entier1 – écrire le code pour le bouton Quitter
 - 4. Écrire du code de sorte que Calculer ne s'active que quand il y a des valeurs dans les zones de saisie entier1 et entier2
 - 5. Écrire du code de sorte que dans entier1 et entier2 on n'autorise que les touches 0 à 9, ← (Backspace) et ↵ (Enter)
 - 6. Écrire du code pour afficher un message en cas de saisie de 0 dans entier2 pour les options pgcd et reste de la division